



ECOLE MAROCAINE DES SCIENCES DE L'INGENIEUR

Annee Universitaire : 2025 / 2026

Projet de Fin d'Annee (PFA)

MEDICAL BIG DATA PLATFORM

Plateforme Big Data medicale pour la collecte, le traitement, l'analyse et la valorisation conversationnelle des donnees de sante via un pipeline hybride ETL, RAG local, recherche web medicale et assistant intelligent.

Filiere : Ingenierie de l'Intelligence Artificielle et de la Data (IADATA)

Module : Architecture de Donnees

REALISE PAR

- Oussama Mennoun
- Mohammed Mekkaoui
- Rayane Ait Ali

ENCADRE PAR

A renseigner

Rapport academique inspire de la structure de couverture du fichier PDF d'exemple fourni.

1. Resume executif

Medical Big Data Platform est une plateforme de donnees medicales qui combine:

- une collecte de contenus depuis des sources medicales fiables
- une architecture de stockage de type **Bronze / Silver / Gold**
- une orchestration batch avec **Airflow**
- un mode streaming avec **Kafka**
- un stockage objet avec **MinIO**
- un entrepot analytique sous **PostgreSQL**
- une indexation vectorielle locale avec **ChromaDB**
- un chatbot medical informationnel hybride **RAG local + web search + OpenAI**
- un dashboard web pour visualiser les indicateurs du pipeline et l'usage du chatbot

Le projet ne se limite donc pas a un chatbot. Il s'agit d'une mini-plateforme data + IA qui couvre la chaine complete, depuis la collecte de donnees jusqu'a la consommation finale par un utilisateur via une interface web.

2. Contexte et problematique

Le domaine medical contient un volume important d'informations utiles, mais ces contenus sont souvent:

- disperses sur plusieurs sites
- heterogenes en structure
- brutes par des elements de navigation web
- difficiles a convertir en reponses courtes, fiables et comprehensibles

Le besoin auquel repond ce projet est double:

1. Centraliser et preparer des donnees medicales exploitables dans une architecture data moderne.
2. Mettre ces donnees au service d'un assistant conversationnel capable de fournir des reponses structurees, prudentes et sourcables.

Le projet assume une posture informationnelle et non diagnostique. Cette contrainte est visible dans les prompts, les fallbacks et les messages de prudence exposes par le backend.

3. Objectifs du projet

Les objectifs techniques et fonctionnels identifies dans le depot sont les suivants:

- automatiser la collecte de contenus medicaux depuis **MedlinePlus** et **NHS**
- stocker les donnees brutes et transformees dans un data lake **MinIO**
- appliquer une logique de preparation de donnees **Bronze / Silver / Gold**

- produire des indicateurs analytiques exploitables par un dashboard ou un outil BI
- construire un index local pour la recherche RAG
- generer des reponses medicales avec un pipeline hybride **local + web**
- offrir une interface de chat multi-conversation avec memoire locale
- tracer les interactions et collecter du feedback pour une future boucle d'amelioration

4. Perimetre fonctionnel reel du depot

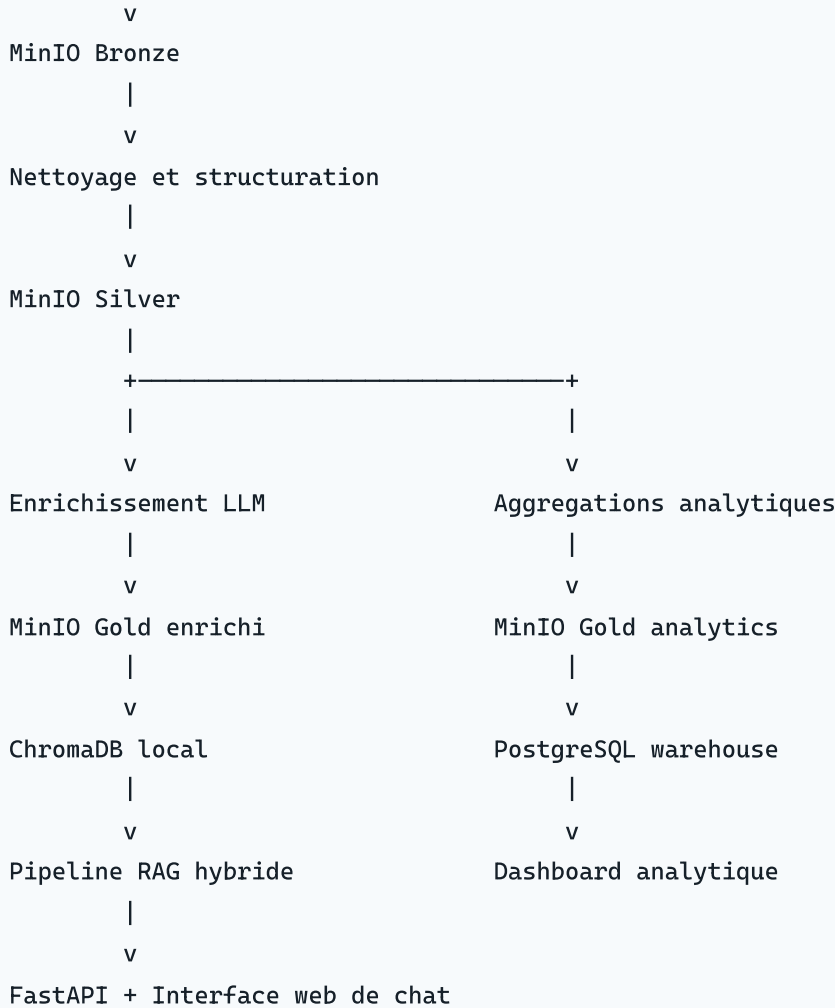
Le depot est organise en plusieurs modules principaux:

- [scraping/scrapper.py](#)
- [scraping/scrapper_medline.py](#)
- [scraping/scrapper_nhs.py](#)
- [kafka/producer.py](#)
- [kafka/consumer.py](#)
- [spark/bronze_to_silver.py](#)
- [spark/llm_enrich_silver.py](#)
- [spark/silver_to_gold.py](#)
- [spark/load_gold_to_postgres.py](#)
- [airflow/dags/pipeline_medical.py](#)
- [chatbot/ingest_documents.py](#)
- [chatbot/rag_pipeline.py](#)
- [chatbot/web_search.py](#)
- [chatbot/llm_generator.py](#)
- [chatbot/hybrid_pipeline.py](#)
- [chatbot/app.py](#)
- [chatbot/index.html](#)
- [dashboard/index.html](#)
- [warehouse/create_tables.sql](#)
- [docker-compose.yml](#)

5. Architecture globale

L'architecture logique du projet peut etre representee ainsi:

```
Sources medicales web
    |
    v
Scraping batch / Streaming Kafka
    |
```



Cette architecture a deux sorties principales:

- une sortie analytique orientee pilotage et visualisation
- une sortie conversationnelle orientee aide informationnelle

6. Infrastructure et services

Le fichier [docker-compose.yml](#) declare les services d'infrastructure suivants:

- `minio` pour le stockage objet
- `postgres` pour le warehouse analytique
- `zookeeper` et `kafka` pour l'ingestion evenementielle
- `airflow-init`, `airflow-webserver` et `airflow-scheduler` pour l'orchestration batch

Les ports exposes montrent clairement le role de chaque service:

- `9000` et `9001` pour MinIO
- `5432` pour PostgreSQL
- `2181` pour Zookeeper
- `9092` pour Kafka

- `8080` pour Airflow

Le compose configure également des variables d'environnement partagées entre les composants, notamment pour MinIO et PostgreSQL.

7. Collecte des données

7.1 Scraping batch

Le scraping batch principal est implémenté dans [scraping/scrapper.py](#).

Le flux est le suivant:

1. Lecture d'une liste d'URLs depuis `scraping/urls_medline.txt` ou `scraping/urls.txt`.
2. Requête HTTP avec entêtes de navigateur.
3. Extraction du titre et du contenu principal.
4. Construction d'un JSON normalisé avec `title`, `content`, `source`, `url`, `scraped_at`.
5. Génération d'un identifiant `md5` à partir de l'URL.
6. Dépôt du résultat dans MinIO bucket `bronze` sous le préfixe `medical_articles/`.

7.2 Scrapers spécialisés

Deux scrapers spécialisés sont présents:

- [scraping/scrapper_medline.py](#)
- [scraping/scrapper_nhs.py](#)

`scrapper_medline.py` joue surtout le rôle d'exemple cible sur une URL MedlinePlus.

`scrapper_nhs.py` est plus robuste:

- lecture d'URLs depuis `scraping/urls_nhs.txt`
- plusieurs sélecteurs HTML pour s'adapter à des variations de structure
- nettoyage du texte
- sauvegarde locale JSON dans `scraping/output/bronze/`

Cette différence montre une évolution intéressante du projet: la logique de collecte n'est pas monolithique et peut être adaptée par source.

8. Ingestion streaming avec Kafka

Le projet ne repose pas uniquement sur du batch. Il contient aussi une brique streaming.

8.1 Producteur Kafka

Dans [kafka/producer.py](#), le producteur:

- scrape une série d'URLs MedlinePlus

- construit un dictionnaire JSON
- envoie chaque article dans le topic Kafka `medical_articles`

8.2 Consommateur Kafka

Dans [kafka/consumer.py](#), le consommateur:

- écoute le topic `medical_articles`
- deserialize les messages JSON
- regenere un identifiant unique a partir de l'URL
- sauvegarde le contenu dans MinIO bucket `bronze` sous `medical_articles_stream/`

Le flux streaming est donc:

Web → Kafka Producer → Kafka Topic → Kafka Consumer → MinIO Bronze

Cette double strategie `batch + streaming` renforce la valeur pedagogique du projet pour une soutenance.

9. Architecture Medallion

Le projet suit une logique medallion visible dans les buckets et prefixes MinIO.

9.1 Bronze

La couche Bronze contient les donnees brutes ou quasi brutes:

- articles scrapes
- titre
- contenu
- source
- URL
- horodatage de scraping

Origines:

- batch via [scraping/scrapper.py](#)
- streaming via [kafka/consumer.py](#)

9.2 Silver

La transformation `Bronze → Silver` se trouve dans [spark/bronze_to_silver.py](#).

Transformations appliquees:

- suppression du HTML
- nettoyage du bruit specifique a MedlinePlus

- normalisation des caracteres et des espaces
- detection de langue avec `langdetect`
- categorisation simple basee sur le titre et l'URL
- calcul de `content_length`
- validation minimale des enregistrements

Le resultat est ecrit dans le bucket `silver` sous `medical_articles_clean/`.

9.3 Gold analytique

La transformation `Silver → Gold` est implemente dans `spark/silver_to_gold.py`.

Le script calcule:

- `articles_by_source`
- `articles_by_category`
- `top_keywords`
- `global_stats`

Ces objets JSON sont ecrits dans le bucket `gold` sous `analytics/`.

10. Enrichissement LLM de la couche Silver

Un point tres interessant du projet est la presence d'une etape supplementaire d'enrichissement:

- `spark/llm_enrich_silver.py`

Ce script:

- lit les documents `Silver`
- construit un prompt de structuration medicale
- peut recuperer un contexte web complementaire via `chatbot/web_search.py`
- appelle OpenAI ou Azure OpenAI selon la configuration
- produit un JSON enrichi avec:
 - `disease`
 - `summary`
 - `key_symptoms`
 - `causes`
 - `treatments`
 - `medications`
 - `red_flags`
 - `contraindications`
 - `preventive_tips`

- `confidence`
- `language`

Le resultat est stocke dans le bucket `gold` sous `llm_enriched/`.

Cette etape cree un pont tres fort entre la partie ETL et la partie IA generative.

11. Chargement dans le warehouse PostgreSQL

Le chargement des sorties Gold vers PostgreSQL est gere par [spark/load_gold_to_postgres.py](#).

Les tables sont definies dans [warehouse/create_tables.sql](#):

- `articles_by_source`
- `articles_by_category`
- `top_keywords`
- `global_stats`

Le script recharge ces tables en supprimant les anciennes lignes avant insertion. Le warehouse sert ensuite de source privilegiee pour le dashboard.

12. Orchestration avec Airflow

La DAG [airflow/dags/pipeline_medical.py](#) orchestre le pipeline batch.

Ordre des taches:

1. `scrape_batch_articles`
2. `transform_bronze_to_silver`
3. `enrich_silver_with_llm`
4. `transform_silver_to_gold`
5. `create_postgres_tables`
6. `load_gold_to_postgres`

La DAG est planifiee avec l'expression `0 * * * *`, soit une execution horaire.

L'interet de cette orchestration est qu'elle formalise le pipeline de bout en bout et rend la demonstration plus professionnelle.

13. Indexation documentaire pour le RAG

Le fichier [chatbot/ingest_documents.py](#) assure l'alimentation de `ChromaDB`.

Deux modes d'ingestion sont prevus:

- mode principal a partir de `gold/llm_enriched/`
- mode de secours a partir de `silver/medical_articles_clean/`

Le script:

- lit les documents depuis MinIO
- nettoie les textes
- detecte une maladie principale
- detecte une section semantique comme `symptoms`, `causes`, `prevention`, `treatment`
- decoupe les contenus en chunks avec overlap
- elimine les segments trop brutes
- stocke les chunks dans une collection ChromaDB persistante

Les metadonnees ajoutees aux chunks sont utiles pour le ranking:

- `title`
- `category`
- `section`
- `source`
- `url`
- `chunk_index`
- `disease`
- indicateurs d'enrichissement LLM

14. RAG local et recherche d'information

Le moteur local se trouve dans [chatbot/rag_pipeline.py](#).

Il ne se contente pas d'une simple similarite vectorielle. Il ajoute des heuristiques metier:

- detection du sujet principal via `TOPIC_MAP`
- detection de l'intention via `SECTION_HINTS`
- vector search sur ChromaDB
- fallback keyword search si le contexte vectoriel est faible
- reranking par recouvrement lexical et metadonnees

Cette logique permet de mieux isoler des sujets medicaux proches et de limiter les melanges de contexte.

15. Recherche web medicale controlee

La recherche web de secours est implementee dans [chatbot/web_search.py](#).

Caracteristiques principales:

- filtrage sur des domaines medicaux autorises
- correction orthographique rudimentaire pour certains termes medicaux
- detection de condition et d'intention

- score des resultats selon le domaine, le titre, l'URL et le type de page
- extraction de contenu depuis les pages retenues

Les domaines privileges incluent:

- `medlineplus.gov`
- `nhs.uk`
- `who.int`
- `cdc.gov`
- `mayoclinic.org`
- `nih.gov`
- `clevelandclinic.org`
- `msdmanuals.com`

Cette approche renforce la fiabilite des reponses par rapport a une recherche web generaliste.

16. Generation de reponses avec OpenAI

Le module `chatbot/llm_generator.py` gere la generation finale.

Il prend en charge:

- OpenAI standard ou Azure OpenAI
- detection de langue de la question
- adaptation du niveau d'explication via `CHATBOT_LEARNER_PROFILE`
- prompts prudents pour l'assistance medicale
- suppression des URLs et citations inline dans la sortie
- fallback si aucune cle API n'est disponible

Deux modes principaux sont exposes:

- `generate_answer( ... )` pour le contexte local ou web prepare
- `generate_web_answer( ... )` pour la recherche web outillee par OpenAI

Le systeme insiste sur plusieurs garde-fous:

- ne pas diagnostiquer
- ne pas inventer de faits
- rester prudent face a des symptomes personnels
- conserver la langue de l'utilisateur

17. Pipeline hybride de decision

Le coeur decisionnel du chatbot est `chatbot/hybrid_pipeline.py`.

Sa mission est de choisir entre plusieurs routes:

- `local_rag`
- `web_search`
- `local_rag_fallback`
- `no_context`

Le pipeline prend aussi en charge:

- la detection de follow-up ambigus
- la reconstruction de la question effective
- la priorisation du web pour des narratifs personnels de symptomes
- le calcul d'un niveau de confiance
- la generation de diagnostics de route utiles au debug et a la demo

Ce composant est l'une des parties les plus abouties du projet, car il relie retrieval, generation, prudence clinique et experience utilisateur.

18. Backend FastAPI

L'application est exposee via [chatbot/app.py](#).

Endpoints principaux:

- `GET /` pour l'interface de chat
- `GET /dashboard` pour le dashboard analytique
- `GET /health` pour l'etat du service
- `GET /api/dashboard` pour les donnees du dashboard
- `GET /api/chatbot-metrics` pour les metriques du chatbot
- `GET /api/learning/summary` pour la boucle d'apprentissage
- `POST /feedback` pour enregistrer une evaluation
- `POST /api/learning/export` pour exporter un dataset JSONL
- `POST /ask` pour poser une question au chatbot

L'endpoint `/ask` enregistre en plus:

- la latence
- le mode de reponse choisi
- le niveau de confiance
- les sources utilisees
- l'interaction pour apprentissage futur

19. Interface web du chatbot

Le frontend de chat est integre dans [chatbot/index.html](#).

Fonctionnalites observees dans le code:

- interface monopage responsive
- multi-chat
- stockage local des conversations dans `localStorage`
- affichage structure des reponses
- badge d'etat API
- affichage du modele actif
- saisie multiline avec `Enter` pour envoyer
- cartes de sources separees du texte de reponse
- rendu des niveaux de confiance et du mode selectionne

L'interface est soignee, moderne et clairement au-dessus d'un simple prototype HTML minimal.

20. Dashboard analytique

Le projet contient egalement un dashboard web dans [dashboard/index.html](#).

Ce dashboard consomme `GET /api/dashboard` et affiche:

- total d'articles
- longueur moyenne
- nombre de sources
- nombre de categories
- repartition des articles par source
- repartition par categorie
- top mots-cles
- avertissements sur l'etat des donnees

Le backend de donnees du dashboard est [chatbot/dashboard_data.py](#).

Il privilegie PostgreSQL, puis bascule vers MinIO en fallback si le warehouse est indisponible.

21. Metriques runtime et boucle d'apprentissage

Le projet integre deux briques souvent absentes dans les PFA de ce type.

21.1 Metriques d'usage

[chatbot/metrics_store.py](#) enregistre:

- nombre total de questions
- latence moyenne
- taux d'usage OpenAI
- distribution des modes
- distribution des niveaux de confiance
- langues utilisees
- sujets detectes
- types de sources

21.2 Boucle de feedback

[chatbot/learning_store.py](#) enregistre:

- les interactions dans `chatbot/learning_data/interactions.jsonl`
- les feedbacks dans `chatbot/learning_data/feedback.jsonl`

Il permet ensuite:

- de calculer un resume d'apprentissage
- de selectionner les meilleures interactions selon une note minimale
- d'exporter un dataset JSONL pour un fine-tuning futur

Ce choix donne au projet une perspective d'amelioration continue tres pertinente pour une soutenance.

22. Flux de donnees de bout en bout

Le fonctionnement complet peut etre resume en deux grandes chaines.

22.1 Chaîne data engineering

1. Les articles sont collectes depuis le web.
2. Les donnees brutes sont stockees dans MinIO Bronze.
3. Les enregistrements sont nettoyes vers Silver.
4. Les articles Silver sont enrichis ou agreges vers Gold.
5. Les indicateurs Gold sont charges dans PostgreSQL.
6. Le dashboard lit PostgreSQL ou MinIO.

22.2 Chaîne conversationnelle

1. Les documents Silver ou Gold enrichis sont indexes dans ChromaDB.
2. L'utilisateur pose une question via le frontend.
3. Le backend choisit entre RAG local et recherche web.
4. Le LLM genere une reponse prudente et structuree.
5. Les sources, metriques et feedbacks sont traces.

23. Choix techniques et justification

Les choix techniques visibles dans le depot sont coherents avec les objectifs:

- **MinIO** simule un data lake simple a deployer localement
- **Kafka** apporte une dimension streaming
- **Airflow** formalise l'orchestration batch
- **PostgreSQL** joue le role de warehouse analytique
- **ChromaDB** sert d'index vectoriel local simple pour du RAG
- **FastAPI** permet de publier rapidement une API claire
- **OpenAI** apporte la generation et, selon le mode, la recherche web assistee

L'ensemble donne un projet transversal entre data engineering, backend et IA generative.

24. Forces du projet

Les principales forces du projet sont:

- couverture de bout en bout, de la collecte a l'interface finale
- coexistence de batch et de streaming
- architecture medallion claire
- separation entre analytique et conversationnel
- pipeline hybride local + web plus mature qu'un RAG basique
- prise en compte de la prudence clinique
- interface web soignee
- presence d'un dashboard exploitable
- metriques d'usage et boucle de feedback

Pour un PFA, cette combinaison est tres convaincante, car elle montre a la fois de l'ingenierie data, de l'architecture applicative et de l'IA appliquee.

25. Limites et constats critiques

L'analyse du depot fait aussi apparaitre plusieurs limites importantes.

25.1 Le dossier **spark/** ne contient pas de vrai traitement Spark

Malgre son nom, le dossier **spark/** n'utilise pas **SparkSession** ni **PySpark**. Les scripts sont des traitements Python classiques executes localement sur des objets MinIO.

Conclusion:

- la logique ETL existe bien
- mais l'execution distribuee Apache Spark n'est pas encore reellement implemente

25.2 Couverture documentaire locale encore limitee

Le RAG depend de la richesse des documents indexes. Avec peu de sources et peu d'articles, certaines questions sortiront du perimetre local et basculeront rapidement vers le web.

25.3 Dependance a OpenAI pour le meilleur niveau de reponse

Sans cle OpenAI, le systeme reste utilisable, mais avec des fallbacks plus simples et moins puissants.

25.4 Incoherences de documentation

Le depot montre quelques ecarts entre code et documentation:

- certaines docs mentionnent un `scraper_who.py` qui n'est pas present dans les fichiers analyses
- [dashboard/README.md](#) indique qu'il n'y a pas encore de dashboard integre, alors qu'un dashboard web existe bien dans [dashboard/index.html](#)
- certaines docs parlent d'un projet en cours de finalisation alors que plusieurs briques sont deja fonctionnelles

25.5 Industrialisation encore partielle

Quelques indices montrent que l'industrialisation peut etre renforcee:

- [Dockerfile](#) est vide
- `scraping/requirements.txt` est vide
- il n'y a pas de suite de tests automatisee visible dans le depot
- les conversations sont persistees cote navigateur et non dans une base applicative

26. Risques techniques et fonctionnels

Les risques principaux a mentionner dans un rapport ou a l'oral sont:

- variation de structure HTML des sites scrapes
- qualite variable des contenus selon les sources
- dependance reseau pour la recherche web
- dependance a des credentials API pour les fonctions LLM
- risque de reponses insuffisantes sur des sujets medicaux peu couverts
- risque de confusion si la base locale n'est pas reingeree apres modification des donnees

27. Pistes d'amelioration

Les evolutions les plus pertinentes seraient:

- integrer de vraies transformations distribuees avec `PySpark`
- augmenter le nombre de sources medicales fiables
- enrichir la couche Gold avec des indicateurs plus avances

- persister les conversations et feedbacks dans PostgreSQL
- ajouter une authentification et une gestion d'utilisateurs
- creer une evaluation automatique des reponses RAG
- ajouter une suite de tests unitaires et d'integration
- completer la containerisation de l'application
- introduire un vrai systeme de citations par passage
- relier la boucle de feedback a un workflow de fine-tuning ou de preference learning

28. Valeur academique et professionnelle du projet

Ce projet est pertinent academiquement car il mobilise plusieurs competences complementaires:

- collecte de donnees web
- stockage data lake
- ETL
- orchestration
- streaming
- warehouse
- recherche d'information
- LLM engineering
- backend web
- UX frontend

Professionnellement, il ressemble a une plateforme exploratoire credibilisant un cas d'usage concret: la mise a disposition d'informations medicales structurees via une experience conversationnelle controlee.

29. Conclusion

Medical Big Data Platform est un projet riche, transversal et deja tres presentable.

Sa force principale est d'assembler dans un meme depot:

- une chaine de donnees medicales **batch + streaming**
- une architecture **Bronze / Silver / Gold**
- un entrepot analytique
- un moteur RAG local
- une recherche web medicale filteree
- un chatbot hybride avec memoire, sources et garde-fous
- un dashboard analytique
- une boucle de feedback orientee amelioration continue

Le projet n'est pas encore totalement industrialise et certaines briques restent perfectibles, notamment la veritable integration Spark, les tests et la couverture documentaire. Malgre cela, l'ensemble constitue deja une base solide pour un rapport de PFA et pour une soutenance technique convaincante.